

Complete EKS Cluster Setup Documentation (Manual AWS Console + ALB Controller)

Overview

This documentation covers the full setup process for a production-style Amazon EKS cluster using:

- AWS Console for cluster creation
- Managed Node Groups
- AWS Load Balancer Controller
- EBS CSI Driver
- IRSA (IAM Roles for Service Accounts)
- Helm
- kubectl

This setup enables:

- Kubernetes workloads on AWS
- Dynamic EBS volume provisioning
- Automatic ALB creation using Kubernetes Ingress
- Pod-level routing using AWS Load Balancer Controller

Architecture Overview

```
Laptop (kubectl)
  ↓
EKS Control Plane
  ↓
Worker Nodes (EC2)
  ↓
Pods / Containers
  ↓
Ingress
  ↓
AWS Load Balancer Controller
  ↓
AWS Application Load Balancer (ALB)
```

Prerequisites

Install the following tools:

AWS CLI

<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

Verify:

```
aws --version
```

kubectl

<https://kubernetes.io/docs/tasks/tools/>

Verify:

```
kubectl version --client
```

eksctl

<https://eksctl.io/installation/>

Verify:

```
eksctl version
```

Helm

<https://helm.sh/docs/intro/install/>

Verify:

```
helm version
```

Configure AWS CLI

```
aws configure
```

Enter:

- AWS Access Key
- AWS Secret Key
- Region: us-east-1
- Output format: json

Verify:

```
aws sts get-caller-identity
```

PART 1 — Create EKS Cluster IAM Role

Go to:

AWS Console → IAM → Roles → Create Role

Settings

Trusted Entity

- AWS Service

Use Case

- EKS
- EKS - Cluster

Permissions

Attach:

```
AmazonEKSClusterPolicy
```

Role Name

```
eks-cluster-role
```

Create role.

PART 2 — Create EKS Cluster Using AWS Console

Go to:

AWS Console → EKS → Clusters → Create Cluster

Choose

Custom Configuration

Cluster Configuration

Cluster Name

banking-cluster

Kubernetes Version

Use latest stable version.

Cluster IAM Role

Select:

eks-cluster-role

Authentication Mode

EKS API

Auto Mode Compute

Disabled.

Networking Configuration

VPC

Select your custom VPC.

Subnets

Select:

- Public subnets

- Private subnets

Endpoint Access

Public and private

Add-ons

Keep default:

- VPC CNI
- CoreDNS
- kube-proxy

Create cluster.

Cluster creation takes approximately:

10–15 minutes

PART 3 — Create Node Group

Go to:

EKS → banking-cluster → Compute → Add Node Group

Create Node Group IAM Role

Go to:

IAM → Roles → Create Role

Trusted Entity

- AWS Service
- EC2

Attach Policies

AmazonEKSWorkerNodePolicy
AmazonEKS_CNI_Policy
AmazonEC2ContainerRegistryReadOnly

Role Name

eks-node-group-role

Create role.

Node Group Settings

Node Group Name

banking-node-group

Node IAM Role

eks-node-group-role

Capacity Type

On-Demand

AMI Type

Amazon Linux 2023

Instance Type

t3.small

Scaling

Desired:

1

Minimum:

1

Maximum:

2

Subnets

IMPORTANT:

Choose ONLY private subnets.

Do NOT select public subnets.

Create node group.

Why Pods Get IPs Inside EKS

AWS VPC CNI gives pods real VPC IP addresses.

That is why:

ALB → Pod IPs

works.

Pods receive secondary private IPs from the EC2 node ENI.

PART 4 — Connect kubectl To Cluster

Run:

```
aws eks update-kubeconfig --region us-east-1 --name banking-cluster
```

Verify:

```
kubectl get nodes
```

Expected:

```
STATUS = Ready
```

PART 5 — Install EBS CSI Driver

The EBS CSI Driver enables Kubernetes to dynamically create AWS EBS volumes.

Without this:

- Persistent Volumes will not work
- PostgreSQL storage will fail

Install through:

EKS → Add-ons → Amazon EBS CSI Driver

Verify:

```
kubectl get pods -n kube-system
```

Look for:

```
ebs-csi-controller  
ebs-csi-node
```

PART 6 — Install AWS Load Balancer Controller

The AWS Load Balancer Controller automatically creates:

- ALBs
- Target Groups
- Listener Rules
- Pod Registrations

based on Kubernetes Ingress resources.

Step 1 — Download IAM Policy

```
curl -o iam_policy.json https://raw.githubusercontent.com/kubernetes-sigs/
aws-load-balancer-controller/main/docs/install/iam_policy.json
```

Step 2 — Create IAM Policy

```
aws iam create-policy --policy-name AWSLoadBalancerControllerIAMPolicy --
policy-document file://iam_policy.json
```

Step 3 — Associate OIDC Provider

IRSA requires OIDC.

Run:

```
eksctl utils associate-iam-oidc-provider --region us-east-1 --cluster
banking-cluster --approve
```

This creates the trust bridge between:

Kubernetes Service Accounts ↔ AWS IAM Roles

Step 4 — Create IAM Service Account

```
eksctl create iamserviceaccount --cluster=banking-cluster --namespace=kube-
system --name=aws-load-balancer-controller --attach-policy-arn=arn:aws:iam::
482112969602:policy/AWSLoadBalancerControllerIAMPolicy --override-existing-
serviceaccounts --region us-east-1 --approve
```

This creates:

- Kubernetes service account
- IAM role
- IAM trust relationship

Step 5 — Install Helm Repo

```
helm repo add eks https://aws.github.io/eks-charts
```

Update repo:

```
helm repo update
```

Step 6 — Get VPC ID

```
aws eks describe-cluster --name banking-cluster --query  
"cluster.resourcesVpcConfig.vpcId" --output text
```

Step 7 — Install AWS Load Balancer Controller

```
helm install aws-load-balancer-controller eks/aws-load-balancer-controller ^  
-n kube-system ^  
--set clusterName=banking-cluster ^  
--set serviceAccount.create=false ^  
--set serviceAccount.name=aws-load-balancer-controller ^  
--set region=us-east-1 ^  
--set vpcId=vpc-0fc2506613a02879c
```

Verify Controller Installation

```
kubectl get pods -n kube-system
```

Look for:

```
aws-load-balancer-controller
```

Status should be:

```
Running
```

How ALB Routing Works In EKS

Ingress YAML does not directly create ALBs.

The process is:

```
Ingress YAML
      ↓
AWS Load Balancer Controller
      ↓
AWS APIs
      ↓
ALB + Target Groups + Listener Rules
```

The controller watches:

- Ingress
- Services
- Endpoints
- Pods

Target groups contain:

```
Pod IPs
```

NOT EC2 instances.

That is why target type appears as:

```
IP
```

inside AWS Target Groups.

PART 7 — Clone Banking Application

```
git clone https://github.com/Yampss/banking-eks-dmo.git
```

Go to project:

```
cd banking-eks-dmo
```

PowerShell Execution Policy

Windows blocks PowerShell scripts by default.

Enable local script execution:

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Press:

```
Y
```

PART 8 — Deploy Application

Apply Namespace

```
kubectl apply -f k8s\namespace.yaml
```

Apply StorageClass

```
kubectl apply -f k8s\storageclass.yaml
```

Apply ConfigMaps

```
kubectl apply -f k8s\postgres-init-configmap.yaml  
kubectl apply -f k8s\configmap.yaml
```

Apply Secrets

```
kubectl apply -f k8s\secrets.yaml
```

Deploy PostgreSQL

```
kubectl apply -f k8s\postgres.yaml
```

Wait for readiness:

```
kubectl wait --for=condition=ready pod -l app=postgres -n banking --  
timeout=180s
```

Deploy Backend Services

```
kubectl apply -f k8s\user-service.yaml  
kubectl apply -f k8s\account-service.yaml  
kubectl apply -f k8s\transaction-service.yaml
```

Deploy Frontend

```
kubectl apply -f k8s\frontend.yaml
```

Deploy Ingress

```
kubectl apply -f k8s\ingress.yaml
```

This automatically triggers:

```
Ingress  
  ↓  
AWS Load Balancer Controller  
  ↓  
ALB Creation
```

Verify Deployment

Check Pods

```
kubectl get pods -n banking
```

Check Ingress

```
kubectl get ingress -n banking
```

Initially:

```
ADDRESS = empty
```

After a few minutes:

```
k8s-banking-xxxxx.us-east-1.elb.amazonaws.com
```

Open the ALB DNS in browser.

Final Architecture

```
graph TD
    Internet --> AWS_ALB[AWS ALB]
    AWS_ALB --> Listener_Rules[Listener Rules]
    Listener_Rules --> Target_Groups[Target Groups]
    Target_Groups --> Pod_IPs[Pod IPs]
    Pod_IPs --> Containers
```

Important Kubernetes Concepts Learned

EKS Control Plane

Managed by AWS.

Contains:

- API Server
 - Scheduler
 - etcd
 - Controller Manager
-

Node Group

Collection of EC2 worker nodes.

Runs:

- Pods
 - Containers
 - kubelet
-

AWS VPC CNI

Gives pods real VPC IP addresses.

EBS CSI Driver

Enables dynamic EBS provisioning for Kubernetes Persistent Volumes.

AWS Load Balancer Controller

Automatically manages:

- ALBs
- Listener rules
- Target groups
- Pod registrations

based on Kubernetes Ingress.

IRSA

IAM Roles for Service Accounts.

Securely allows Kubernetes pods to assume AWS IAM roles.

No static AWS credentials required.

Cleanup

Delete cluster:

```
eksctl delete cluster --name banking-cluster --region us-east-1
```

Delete leftover:

- EBS volumes
- IAM roles
- Load balancers
- Target groups

from AWS Console to avoid charges.